测试计划

1 黑盒测试计划

1.1 黑盒测试目标

黑盒测试重点检查主要功能、角色权限和核心业务流程是否符合需求。具体目标如下：

1. 验证系统主要功能是否与需求描述一致。
2. 验证教师、学生两类角色的功能权限是否正确。
3. 验证课程平台核心业务流程能否端到端顺利完成。
4. 基于等价类、边界值、决策表和场景法设计测试用例。
5. 使用自动化接口测试与少量人工页面测试相结合的方式完成验证。

1.2 黑盒测试范围与重点

被测对象为当前仓库中的 `RAG-Knowledge-Assistant` 项目。该项目包含前端与后端两部分，功能覆盖用户管理、课程管理、课程资料管理、练习题管理和课程社区管理，满足课程作业对项目规模和功能点数量的要求。

本次黑盒测试覆盖以下功能点：

1. 用户注册
2. 用户登录
3. 个人信息查看
4. 个人信息修改
5. 教师创建课程
6. 学生加入课程
7. 学生退出课程
8. 课程资料上传

9. 教师解散课程
10. 课程资料查看
11. 课程资料下载
12. 课程资料 PDF 预览
13. 课程资料删除
14. 练习题上传
15. 练习题查看
16. 练习题作答
17. 作答结果反馈
18. 课程社区发帖
19. 课程社区评论
20. 课程社区回复
21. 核心流程黑盒验证

重点检查以下内容：

1. 不同角色的权限控制是否正确。
2. 各业务操作后的页面表现与数据状态是否一致。
3. 文件相关功能是否正确处理格式、大小限制和异常情况。
4. 练习题与课程社区功能是否能在课程上下文中正确关联。
5. 核心业务流程中的前后步骤是否能够正确衔接。

核心流程定义为：

教师登录 → 创建课程 → 上传资料 → 上传练习题 → 学生登录 → 加入课程 → 查看资料 → 完成练习 → 获得反馈 → 参与课程社区发帖 / 评论 / 回复

1.3 黑盒测试方法

测试条件和测试用例主要采用以下方法设计：

1.3.1 等价类划分

将输入条件与业务状态划分为有效等价类和无效等价类，主要包括：

1. 角色：教师、学生、非法角色。
2. 登录信息：正确账号密码、错误密码、未注册账号、空输入。
3. 文件类型：PDF、非 PDF、ZIP、超限文件。
4. 课程状态：未加入、已加入、已解散、非本人课程。

1.3.2 边界值分析

重点覆盖以下边界条件：

1. 分页页码边界。
2. 文件大小上限边界。
3. 标题、描述、评论内容等空输入边界。
4. ZIP 压缩包单层目录与多层目录边界。
5. 练习题集题目数量边界。

1.3.3 决策表法

主要用于角色权限和条件组合较明显的场景，例如：

1. 注册是否允许提交。
2. 登录是否成功。
3. 创建课程是否允许执行。
4. 加入课程、退出课程、解散课程是否满足角色与状态条件。
5. PDF 预览与评论回复是否满足前置条件。

1.3.4 场景法

用于验证跨模块核心流程，检查真实教学场景下业务操作能否连续完成，以及模块之间的数据关联与状态流转是否正确。

1.4 黑盒测试环境、工具与数据

1.4.1 测试环境

1. 操作系统：Windows 11 开发环境。
2. 前端：React 19。
3. 后端：FastAPI。
4. 浏览器：Chrome 与 Edge。
5. 前端访问地址：`http://localhost:3000`。
6. 后端接口地址：`http://127.0.0.1:8000`。

1.4.2 测试工具

1. `Codex`：用于辅助梳理需求、编写测试计划、设计测试用例、生成自动化测试代码和整理测试结果。
2. `pytest + FastAPI TestClient`：用于执行自动化黑盒接口测试。
3. 浏览器人工操作：用于补充自动化测试难以完全覆盖的页面交互、PDF 预览、下载行为和页面级流程验证。

1.4.3 测试数据

1. 教师账号 1 个。
2. 学生账号 1 个。
3. 测试课程若干。
4. 测试资料若干，包含 PDF、非 PDF、ZIP 等文件。
5. 测试练习题集若干，覆盖单选、多选、填空三类题型。

1.5 黑盒测试执行安排

黑盒测试采用“自动化接口测试为主、少量人工页面测试为辅”的方式执行。自动化测试承担主体覆盖任务，人工测试主要补充前端页面交互、文件下载与 PDF 预览等浏览器行为验证。

执行顺序如下：

1. 环境与测试数据准备。
2. 用户管理模块测试。
3. 课程管理模块测试。
4. 课程资料管理模块测试。
5. 练习题管理模块测试。
6. 课程社区管理模块测试。
7. 核心流程端到端测试。
8. 回归验证与结果整理。

测试优先级划分如下：

1. `P0`：用户登录、教师创建课程、学生加入课程、资料上传、练习题查看与作答、核心流程贯通。
2. `P1`：用户注册、个人信息修改、资料下载、PDF 预览、社区发帖评论回复。
3. `P2`：课程搜索分页、资料列表分页、帖子搜索分页等辅助功能。

2 白盒测试计划

2.1 白盒测试目标

白盒测试重点检查成员 B 负责业务在后端完整调用链中的分支处理、路径可达性与数据定义使用关系是否正确。具体目标如下：

1. 以“模块一体化测试”为原则，在同一用例中联合验证 `controller -> service -> dao -> repository` 调用链一致性。
2. 覆盖登录认证、用户信息、课程管理、课程资料管理四类业务在各层的关键逻辑。
3. 基于代码覆盖、基本路径覆盖和数据流分析三种方法设计并执行测试用例。
4. 通过 `pytest + coverage` 统计覆盖率，确保本人负责模块对应后端代码总体覆盖率不低于 `95%`。
5. 使用 `Codex` 辅助完成测试用例补全、覆盖率分析和测试报告生成，形成智能化测试工作流。

2.2 白盒测试范围与被测代码模块

本次白盒测试范围限定为成员 B 负责业务对应的后端模块代码，覆盖用户与认证、课程管理、课程资料管理三类业务。

被测后端模块代码如下（包含控制、服务与数据访问实现）：

1. `Code/backend/controller/LoginController.py`
2. `Code/backend/controller/UserController.py`
3. `Code/backend/controller/CourseController.py`
4. `Code/backend/controller/rag_controller.py`
5. `Code/backend/service/LoginService.py`
6. `Code/backend/service/UserService.py`
7. `Code/backend/service/CourseService.py`
8. `Code/backend/dao/document_dao.py`
9. `Code/backend/repository/UserRepository.py`
10. `Code/backend/repository/UserAuthRepository.py`
11. `Code/backend/repository/CourseRepository.py`
12. `Code/backend/repository/document_repository.py`

覆盖目标如下：

1. 一体化覆盖：同一业务用例贯通接口处理、业务决策、数据访问与状态变更。
2. 关键业务分支（成功、失败、权限拒绝、资源不存在、重复操作、文件超限）均被执行。
3. 相关后端代码总体覆盖率不低于 95%。

2.3 白盒测试方法

测试用例主要按照以下三类白盒方法生成并执行：

2.3.1 代码覆盖分析

以代码覆盖率为核心指标，采用“控制器输入驱动”方式组织覆盖：通过接口请求触发完整后端调用链 `controller -> service -> dao -> repository`，仅对 Mongo/GridFS 等外部系统依赖使用替身隔离。按业务链路规划覆盖点如下：

1. 登录认证链路：注册、登录、鉴权、改密、资料更新及对应异常返回分支。
2. 课程管理链路：角色权限、课程存在性、课程状态（正常/已解散）、加入/退出/解散与持久化分支。
3. 资料管理链路：上传类型判断（普通文件/ZIP）、大小限制、存储访问、下载与预览分支。

2.3.2 基本路径覆盖

对条件组合较多的接口流程进行路径拆分，确保关键独立路径至少执行一次，并兼顾成功路径、失败路径与异常路径。重点路径包括：

1. 登录链路：用户存在/不存在、密码匹配/不匹配、无效 token、用户不存在时鉴权失败。
2. 用户资料链路：旧密码正确/错误、认证信息存在/不存在、资料更新成功/异常。
3. 课程链路：创建成功/权限拒绝、加入成功/重复加入/课程不存在/课程已解散、退出成功/未加入/课程不存在、解散成功/无权限/已解散/课程不存在。
4. 资料链路：普通上传成功、ZIP 单层目录上传成功、ZIP 嵌套目录拒绝、单文件超限、批量总大小超限、下载/预览/删除的成功与文件不存在异常。

2.3.3 数据流分析

围绕关键变量的定义（Def）与使用（Use）关系构造测试，重点关注“接口参数定义 → 服务处理 → 数据访问使用 → 异常返回”的变量传播一致性。主要包括：

1. 用户身份与角色变量（`current_user.id`、`current_user.role`）在鉴权、权限判断与后续业务调用中的使用关系。
2. 认证数据（`hashed_password`）在生成、存储、校验、更新过程中的定义与使用关系。

3. 课程标识（`course_id`）在查询、加入、退出、解散流程中的跨层传递关系与异常中断路径。
4. 文件标识（`file_id`）在下载、预览、删除流程中的传递与读取关系。
5. 文件大小相关变量（单文件大小、总大小）在上传流程中的累计比较与阈值判断关系。

2.4 白盒测试工具与数据

2.4.1 测试环境

1. 运行环境：Python 3.11。
2. 后端框架：FastAPI。
3. 关系型数据测试环境：SQLite 内存数据库（测试会话）。
4. 文档存储测试环境：Mongo/GridFS 外部依赖替身。

2.4.2 测试工具

1. `Codex`：辅助设计测试用例、分析未覆盖路径和整理报告。
2. `pytest`：执行自动化白盒测试。
3. `FastAPI TestClient`：通过接口请求驱动后端完整调用链。
4. `pytest-cov`：统计覆盖率并输出未命中行。

2.4.3 测试数据

1. 教师与学生模拟用户对象各 1 组。
2. 课程对象、课程列表和分页参数模拟数据若干。
3. 文档上传测试文件：PDF、TXT、ZIP（单层目录/嵌套目录）等样本、小于限制或超过限制的样本。

2.5 白盒测试执行安排

白盒测试按“模块一体化执行、覆盖率补齐、回归验证”的方式执行，每个模块直接走完整后端调用链。

执行顺序如下：

1. 环境准备与依赖安装（`pytest`、`coverage`）。
2. 核心业务模块一体化测试：分别对用户认证、课程管理、资料管理等模块执行完整后端调用链测试。
3. 数据链路一致性验证：针对每个核心业务模块，自动化验证接口输入、数据库变更、接口输出三者之间的数据一致性与状态同步正确性。
4. 覆盖率统计、薄弱路径定位与用例增补。
5. 回归执行并固化测试结果。
6. 生成测试报告。

覆盖率统计命令示例：

```
pytest tests/ -q --cov=controller --cov=service --cov=repository --cov=dao
--cov-report=term-missing
```

2.6 白盒测试风险与说明

1. 一体化测试中部分数据库与文件存储行为通过替身对象模拟，需在集成环境补充真实依赖联调。
2. 文档存储相关逻辑受 MongoDB/GridFS 环境影响，测试环境与部署环境差异可能导致问题。
3. 覆盖率仅反映代码执行比例，不等同于缺陷零风险，需结合路径与数据流结果及其它测试综合评估。